

TCPDump and Socket Programming with Python

Ibnu F. Kurniawan, Ph.D. Pradini Puspitaningayu, Ph.D.
ibnufebry[at]unesa.ac.id

Komputasi Berbasis Jaringan

2024

ToC

1. Tracing over Application-layer Apps
   - HTTP dump
   - DNS Dump
2. Blocking Client-Server
3. Non-blocking Client-Server
   - Process-based
   - Thread-based
4. Forwarding
   - Code PortForwarding
5. QnA

Tracing over Application-layer Apps

HTTP dump

Log

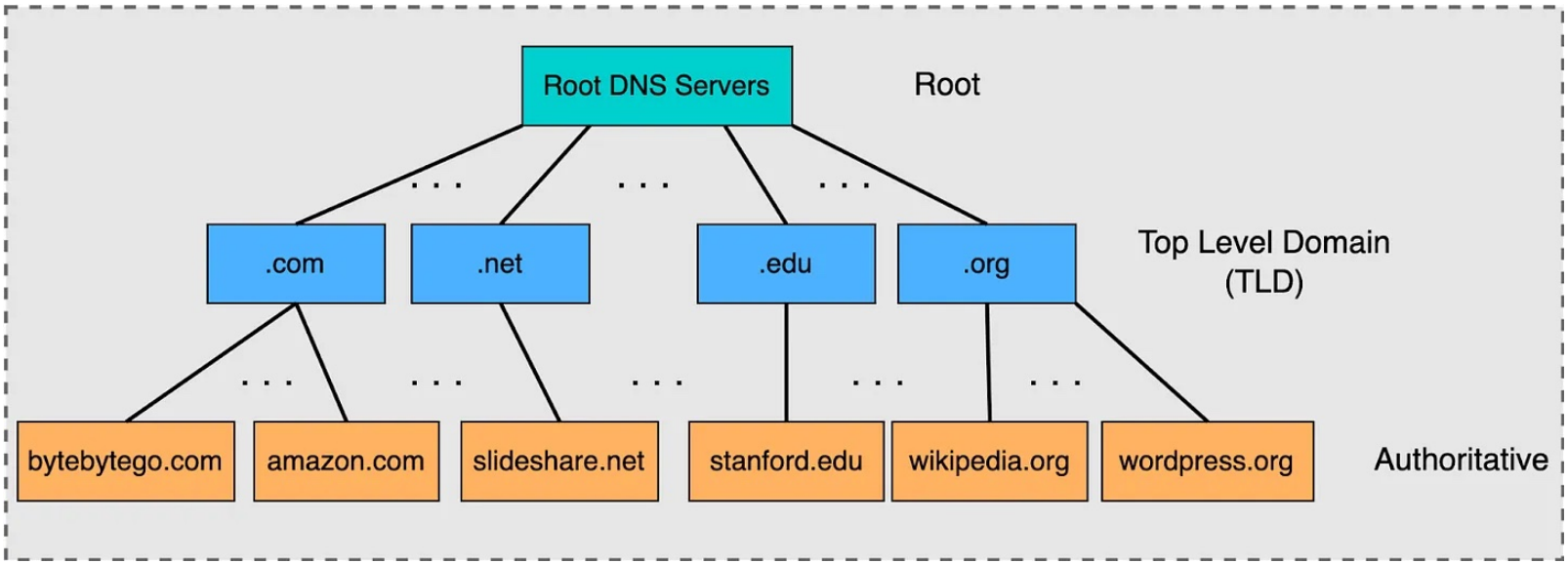
```
1  $ tcpdump -r http.cap
2  reading from file http.cap, link-type EN10MB (Ethernet)
3  17:17:07.311224 IP dialin-145-254-160-237.pools.arcor-ip.net.tip2 > 65.208.228.223.http: Flags [S], seq
   ↪  951057939, win 8760, options [mss 1460,nop,nop,sackOK], length 0
4  17:17:08.222534 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [S.], seq
   ↪  290218379, ack 951057940, win 5840, options [mss 1380,nop,nop,sackOK], length 0
5  17:17:08.222534 IP dialin-145-254-160-237.pools.arcor-ip.net.tip2 > 65.208.228.223.http: Flags [.], ack
   ↪  1, win 9660, length 0
6  17:17:08.222534 IP dialin-145-254-160-237.pools.arcor-ip.net.tip2 > 65.208.228.223.http: Flags [P.], seq
   ↪  1:480, ack 1, win 9660, length 479: HTTP: GET /download.html HTTP/1.1
7  17:17:08.783340 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [.], ack
   ↪  480, win 6432, length 0
8  17:17:08.993643 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [.], seq
   ↪  1:1381, ack 480, win 6432, length 1380: HTTP: HTTP/1.1 200 OK
9  17:17:09.123830 IP dialin-145-254-160-237.pools.arcor-ip.net.tip2 > 65.208.228.223.http: Flags [.], ack
   ↪  1381, win 9660, length 0
10 17:17:09.123830 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [.], seq
   ↪  1381:2761, ack 480, win 6432, length 1380: HTTP
11 17:17:09.324118 IP dialin-145-254-160-237.pools.arcor-ip.net.tip2 > 65.208.228.223.http: Flags [.], ack
   ↪  2761, win 9660, length 0
12 17:17:09.754737 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [.], seq
   ↪  2761:4141, ack 480, win 6432, length 1380: HTTP
13 17:17:09.864896 IP 65.208.228.223.http > dialin-145-254-160-237.pools.arcor-ip.net.tip2: Flags [P.], seq
   ↪  4141:5521, ack 480, win 6432, length 1380: HTTP
```

DNS Dump

DNS' RFC

https://blog.bytebytego.com/p/a-crash-course-in-dns-domain-name

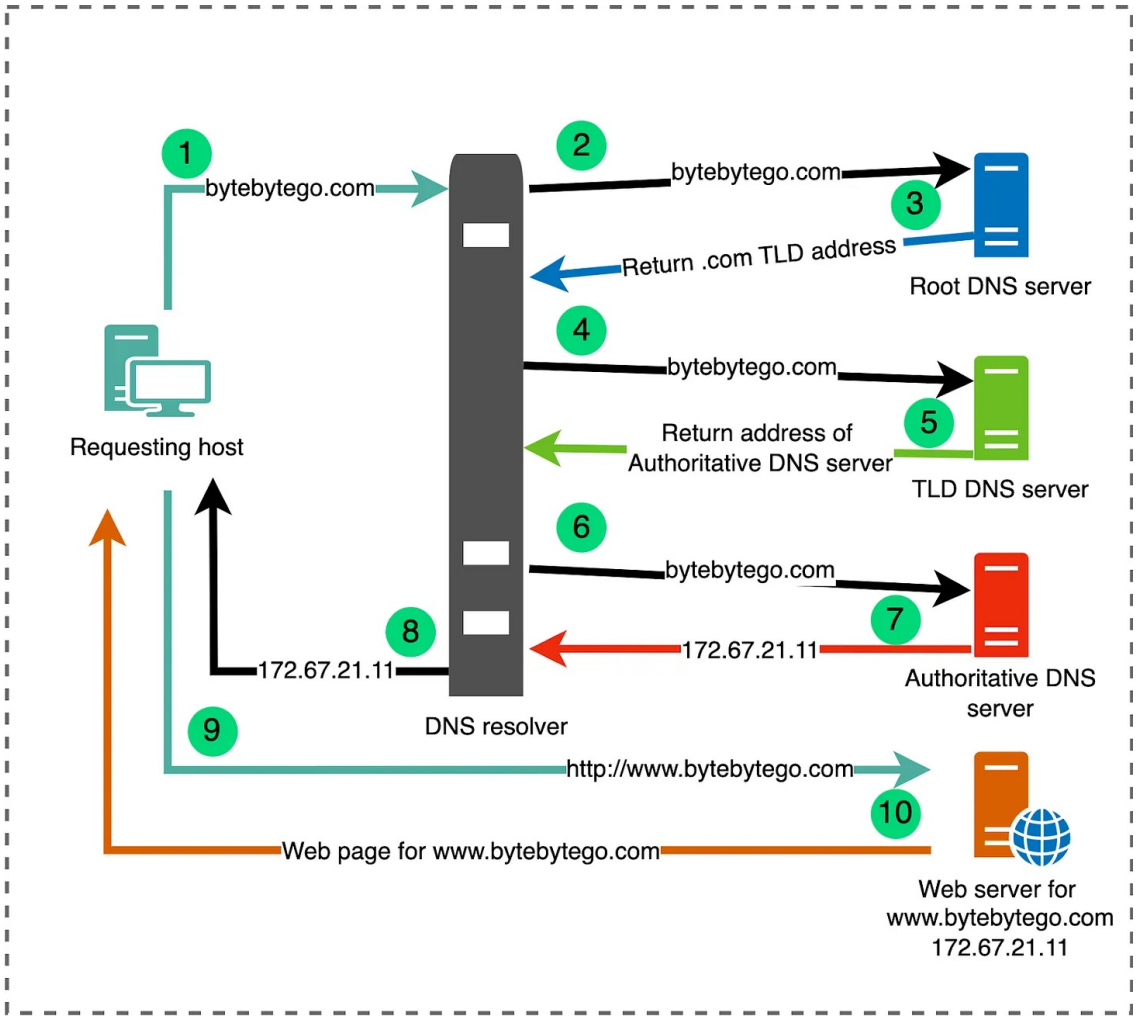
https://www.cloudflare.com/en-gb/learning/dns/dns-records/dns-a-record/



DNS' RFC

https://blog.bytebytego.com/p/a-crash-course-in-dns-domain-name

https://www.cloudflare.com/en-gb/learning/dns/dns-records/dns-a-record/



Dump

```
15:47:46.496046 IP 192.168.170.8.32795 > 192.168.170.20.domain: 4146+ TXT? google.com. (28)
15:47:46.496576 IP 192.168.170.20.domain > 192.168.170.8.32795: 4146 1/0/0 TXT "v=spf1 ptr ?all" (56)
15:47:50.501268 IP 192.168.170.8.32795 > 192.168.170.20.domain: 63343+ MX? google.com. (28)
15:47:51.333401 IP 192.168.170.20.domain > 192.168.170.8.32795: 63343 6/0/6 MX smtp4.google.com. 40, MX
↪  smtp5.google.com. 10, MX smtp6.google.com. 10, MX smtp1.google.com. 10, MX smtp2.google.com. 10, MX
↪  smtp3.google.com. 40 (256)
15:47:59.313231 IP 192.168.170.8.32795 > 192.168.170.20.domain: 18849+ LOC? google.com. (28)
15:47:59.452255 IP 192.168.170.20.domain > 192.168.170.8.32795: 18849 0/0/0 (28)
15:48:07.320873 IP 192.168.170.8.32795 > 192.168.170.20.domain: 39867+ PTR? 104.9.192.66.in-addr.arpa.
↪  (43)
15:48:07.321379 IP 192.168.170.20.domain > 192.168.170.8.32795: 39867 1/0/0 PTR
↪  66-192-9-104.gen.twtelecom.net. (87)
15:49:18.685951 IP 192.168.170.8.32795 > 192.168.170.20.domain: 30144+ A? www.netbsd.org. (32)
15:49:18.734862 IP 192.168.170.20.domain > 192.168.170.8.32795: 30144 1/0/0 A 204.152.190.12 (48)
15:49:35.461181 IP 192.168.170.8.32795 > 192.168.170.20.domain: 61652+ AAAA? www.netbsd.org. (32)
15:49:35.698849 IP 192.168.170.20.domain > 192.168.170.8.32795: 61652 1/0/0 AAAA
↪  2001:4f8:4:7:2e0:81ff:fe52:9a6b (60)
15:50:35.523440 IP 192.168.170.8.32795 > 192.168.170.20.domain: 32569+ AAAA? www.netbsd.org. (32)
15:50:35.523827 IP 192.168.170.20.domain > 192.168.170.8.32795: 32569 1/0/0 AAAA
↪  2001:4f8:4:7:2e0:81ff:fe52:9a6b (60)
15:50:44.735890 IP 192.168.170.8.32795 > 192.168.170.20.domain: 36275+ AAAA? www.google.com. (32)
15:50:44.752428 IP 192.168.170.20.domain > 192.168.170.8.32795: 36275 1/0/0 CNAME www.l.google.com. (52)
15:50:54.349862 IP 192.168.170.8.32795 > 192.168.170.20.domain: 56482+ AAAA? www.l.google.com. (34)
15:50:54.366527 IP 192.168.170.20.domain > 192.168.170.8.32795: 56482 0/0/0 (34)
15:51:35.204348 IP 192.168.170.8.32795 > 192.168.170.20.domain: 48159+ AAAA? www.example.com. (33)
15:51:35.437491 IP 192.168.170.20.domain > 192.168.170.8.32795: 48159 0/0/0 (33)
15:51:46.819984 IP 192.168.170.8.32795 > 192.168.170.20.domain: 9837+ AAAA? www.example.notginh. (37)
```

Blocking Client-Server

Echo Server

```
1 import argparse
2 import socket
3
4 LOCAL_SERVER_HOST = 'localhost'
5 BUFSIZE = 4096
6
7 data_payload = 2048
8 backlog = 5
```

Echo Server

```
10 def echo_server(port):
11     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12     sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
13     server_address = (LOCAL_SERVER_HOST, port)
14     print ("Starting up echo server on %s %s" % server_address)
15
16     sock.bind(server_address)
17     # max. num of queued connections
18     sock.listen(backlog)
19     while True:
20         print("Waiting to receive message from client")
21         client, address = sock.accept()
22         data = client.recv(data_payload)
23
24         if data:
25             print("Data: %s" % data)
26             client.send(data)
27             print("sent %s bytes back to %s" % (data, address))
28
29         client.close()
```

Echo Client

Socket init

```
8  sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9  server_address = (CLIENT_HOST, CLIENT_PORT)
10
11 print(f"Connecting to {CLIENT_HOST} port {CLIENT_PORT}")
12 sock.connect(server_address)
```

Echo Client

Loop

```
14 try:
15     msg = "Test message. This will be echoed"
16     print("Sending %s" % msg)
17
18     sock.sendall(msg.encode('utf-8'))
19
20     amount_rcv = 0
21     amount_exp = len(msg)
22
23     while amount_rcv < amount_exp:
24         data = sock.recv(16)
25         amount_rcv += len(data)
26         print(" Rcv: %s" % data)
```

Non-blocking Client-Server

Process-based

Client

Full code

```
7  class ForkedClient():
8      """
9        A client to test forking server
10     """
11
12     def __init__(self, ip, port):
13         self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
14         self.sock.connect((ip, port))
15
16     def run(self):
17         _pid = os.getpid()
18         print(f"PID {_pid} is sending msg \"{ECHO_MSG}\" to the server")
19
20         _datalength = self.sock.send(bytes(ECHO_MSG, 'utf-8'))
21         print(f"Sent: {_datalength} chars so far")
22
23         _resp = self.sock.recv(BUF_SIZE)
24         print(f"PID {_pid} received msg \"{_resp[5:]}\" from the server")
25
26     def shutdown(self):
27         self.sock.close()
```

Client

Main

```
16 def run(self):
17     _pid = os.getpid()
18     print(f"PID {_pid} is sending msg \"{ECHO_MSG}\" to the
       ↪  server")
19
20     _datalength = self.sock.send(bytes(ECHO_MSG, 'utf-8'))
21     print(f"Sent: {_datalength} chars so far")
22
23     _resp = self.sock.recv(BUF_SIZE)
24     print(f"PID {_pid} received msg \"{_resp[5:]}\" from the
       ↪  server")
```

Client

Init

```
12 def __init__(self, ip, port):
13     self.sock = socket.socket(socket.AF_INET,
       ↪  socket.SOCK_STREAM)
14     self.sock.connect((ip, port))
```

Server

Main parts

```
12 class ForkingServerRequestHandler(socketserver.BaseRequestHandler):
13     def handle(self) -> None:
14         data = str(self.request.recv(BUF_SIZE), 'utf-8')
15         _pid = os.getpid()
16 
17         resp = f"{_pid}: {data}"
18         print(f"Server is sending response [pid:data] =
           ↪  [{resp}]")
19 
20         self.request.send(bytes(resp, 'utf-8'))
21         return
22 
23 class ForkingServer(socketserver.ForkingMixIn,
   ↪  socketserver.TCPServer):
24     pass
```

Main

Main commands

```
11 server = ForkingServer((SERVER_HOST,
   ↪  SERVER_PORT),ForkingServerRequestHandler)
12
13 ip, port = server.server_address
14 server_thread = threading.Thread(target=server.serve_forever)
15 server_thread.daemon = True
16
17 server_thread.start()
18 print(f"Server loop running. PID: {os.getpid()}")
19
20 client1 = ForkedClient(ip, port)
21 client1.run()
22
23 print("Client 1 running.")
```

Main

Closing

```
29 server.shutdown()
30 client1.shutdown()
31 client2.shutdown()
32 server.socket.close()
```

Output

```
$ python3 AsyncComm.py
Server loop running. PID: 37602
PID 37602 is sending msg "Hello server!" to the server
Sent: 13 chars so far
Server is sending response [pid:data] = [37603: Hello server!]
PID 37602 received msg "b': Hello server!'" from the server
Client 1 running.
PID 37602 is sending msg "Hello server!" to the server
Sent: 13 chars so far
Server is sending response [pid:data] = [37604: Hello server!]
PID 37602 received msg "b': Hello server!'" from the server
Client 2 running.

$
```

Thread-based

Your turn

Recipes:

1. `threading`, `socketserver`
2. `socketserver.ThreadingMixIn`

Forwarding

Code PortForwarding

PortForwarder

```
10 class PortForwarder(asyncore.dispatcher):
11     def __init__(self,ip, port, remoteip, remoteport,
       ↪  backlog=5):
12         asyncore.dispatcher.__init__(self)
13         self.remoteip=remoteip
14         self.remoteport=remoteport
15         self.create_socket(socket.AF_INET, socket.SOCK_STREAM)
16         self.set_reuse_addr()
17         self.bind((ip,port))
18         self.listen(backlog)
19 
20     def handle_accept(self):
21         conn, addr = self.accept()
22         print(f"Connected to: {addr}")
23         Sender(Receiver(conn), self.remoteip, self.remoteport)
```

Receiver

```
25 class Receiver(asyncore.dispatcher):
26     def __init__(self, conn):
27         asyncore.dispatcher.__init__(self, conn)
28         self.from_remote_buffer = b""
29         self.to_remote_buffer = b""
30         self.sender = None
31 
32     def handle_connect(self):
33         pass
34 
35     def handle_read(self):
36         read = self.recv(BUFSIZE)
37         self.from_remote_buffer += read
38 
39     def writable(self):
40         return (len(self.to_remote_buffer) > 0)
41 
42     def handle_write(self):
43         sent = self.send(self.to_remote_buffer)
44         self.to_remote_buffer = self.to_remote_buffer[sent:]
45 
46     def handle_close(self):
47         self.close()
48         if self.sender:
49             self.sender.close()
50 
```

Sender

```
51 class Sender(asyncore.dispatcher):
52     def __init__(self, receiver, remoteaddr, remoteport):
53         asyncore.dispatcher.__init__(self)
54         self.receiver = receiver
55         receiver.sender = self
56         self.create_socket(socket.AF_INET, socket.SOCK_STREAM)
57         self.connect((remoteaddr, remoteport))
58 
59     def handle_connect(self):
60         pass
61 
62     def handle_read(self):
63         read = self.recv(BUFSIZE)
64         self.receiver.to_remote_buffer += read
65 
66     def writable(self):
67         return (len(self.receiver.from_remote_buffer) > 0)
68 
69     def handle_write(self):
70         sent = self.send(self.receiver.from_remote_buffer)
71         self.receiver.from_remote_buffer = self.receiver.from_remote_buffer[sent:]
72 
73     def handle_close(self):
74         self.close()
75         self.receiver.close()
```

Main

```
78 if __name__ == "__main__":
79     parser = argparse.ArgumentParser(description="Stackless socket  server example")
80     parser.add_argument("--local-host",action="store",dest="local_host", default=LOCAL_SERVER_HOST)
81     parser.add_argument("--remote-host", action="store", default=REMOTE_SERVER_HOST)
82     parser.add_argument("--local-port", action="store", dest="local_port", type=int, required=True,
       ↪  default=8080)
83     parser.add_argument("--remote-port", action="store", dest="remote_port", type=int, default=443)
84
85     given_args = parser.parse_args()
86     local_host, remote_host = given_args.local_host, given_args.remote_host
87     local_port, remote_port = given_args.local_port, given_args.remote_port
88
89     print(f"Starting port forwarding local {local_host}:{local_port} => remote
       ↪  {remote_host}:{remote_port}")
90
91     PortForwarder(local_host, local_port, remote_host, remote_port)
92     asyncore.loop()
```

Output

```
$ curl localhost:8080
<!DOCTYPE html>
<html lang=en>
  <meta charset=utf-8>
  <meta name=viewport content="initial-scale=1, minimum-scale=1, width=device-width">
  <title>Error 404 (Not Found)!!1</title>
  <style>
    *{margin:0;padding:0}html,code{font:15px/22px
    ↪  arial,sans-serif}html{background:#fff;color:#222;padding:15px}body{margin:7% auto
    ↪  0;max-width:390px;min-height:180px;padding:30px 0 15px}* >
    ↪  body{background:url(//www.google.com/images/errors/robot.png) 100% 5px
    ↪  no-repeat;padding-right:205px}p{margin:11px 0
    ↪  22px;overflow:hidden}ins{color:#777;text-decoration:none}a img{border:0}@media screen and
    ↪  (max-width:772px){body{background:none;margin-top:0;max-width:none;padding-right:0}}#logo{background:u
    ↪  no-repeat;margin-left:-5px}@media only screen and
    ↪  (min-resolution:192dpi){#logo{background:url(//www.google.com/images/branding/googlelogo/2x/googlelogo
    ↪  no-repeat 0% 0%/100%
    ↪  100%;-moz-border-image:url(//www.google.com/images/branding/googlelogo/2x/googlelogo_color_150x54dp.pn
    ↪  0}}@media only screen and
    ↪  (-webkit-min-device-pixel-ratio:2){#logo{background:url(//www.google.com/images/branding/googlelogo/2x
    ↪  no-repeat;-webkit-background-size:100% 100%}}#logo{display:inline-block;height:54px;width:150px}
  </style>
  <a href=//www.google.com/><span id=logo aria-label=Google></span></a>
  <p><b>404.</b> <ins>That’s an error.</ins>
  <p>The requested URL <code>/</code> was not found on this server.  <ins>That’s all we know.</ins>
```

QnA

Tugas

Buatlah implementasi client-server socket yang mensimulasikan "load balancing" secara sederhana dalam `Python`. Detail simulasi adalah sebagai berikut:

1. Server utama (*broker*) menerima *request* dari *client* dan mendistribusikan ke 3 server lain (*worker*) dalam "cluster".
2. Terdapat 2 pendekatan alokasi:
   1. pemerataan jumlah *request* yang sudah dilayani oleh masing-masing server pekerja
   2. berurutan sesuai dengan `server-id` berdasar *counter* milik *broker*
3. Masing-masing worker *server* mempunyai 2 jenis aplikasi yang dilayani (bisa dimisalkan saja (tanpa implementasi):
   1. Long $\Rightarrow$ perhitungan yang kompleks
   2. Short $\Rightarrow$ echo atau perhitungan sederhana
4. Client mengirim pesan *request* ke broker dengan menyebutkan Application ID
   $\Rightarrow$ Silakan definisikan sintaks/protokol/*messaging format*.
5. *Broker* dan *worker* mencetak pesan informasi ke layar.